PORPOISE ROBOTICS

Find the Target

Vehicle Telemetry System

Complete Setup Guide

This guide walks you, step by step, through building and running the whole system — from unboxing to watching your vehicle appear on a live map. No prior coding or electronics experience is assumed. Just follow the steps in order.

WHAT'S INSIDE

PART 01

What you need

HARDWARE CHECKLIST

- [ ] The vehicle, with its Sender ESP32 board, sensors (distance, pressure, GPS) and 16-pixel LED ring already wired on
- [ ] A second ESP32 board (the "Receiver") that plugs into the computer by USB
- [ ] The Freenove ESP32-WROVER camera board
- [ ] A laptop or desktop computer (Windows, Mac, or Linux all work)
- [ ] Two USB cables (one for the receiver, one to program each board in turn)
- [ ] A WiFi network the camera can join (2.4 GHz — the camera cannot use 5 GHz WiFi)

SOFTWARE CHECKLIST (FREE DOWNLOADS)

- [ ] **Arduino IDE —** the program used to load code onto the ESP32 boards
- [ ] **Python 3 —** runs the base station program on the computer

WHAT "FLASHING" MEANS

Every time this guide says "flash" a board, it means: connect that board to the computer with USB, open its program in the Arduino IDE, and click the Upload button. The IDE copies the code onto the board over the cable. It takes 30–90 seconds and you'll see a scrolling progress bar.

PART 02

Install the software

Do this once, on the computer you'll use as the base station. It's also the computer you'll use to flash all three boards.

1 Install the Arduino IDE

Go to **arduino.cc/en/software** and download the free Arduino IDE for your operating system. Install it like any other program.

Open the Arduino IDE, then tell it where to find ESP32 support: go to **File → Preferences** (on Mac: **Arduino IDE → Settings**), find the box labeled **Additional boards manager URLs**, and paste in:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```

Click **OK** to save. Now go to **Tools → Board → Boards Manager**, type **esp32** into the search box, and click Install on the package by **Espressif Systems**. This can take several minutes.

> **IMPORTANT — USE VERSION 3.0.7**
> In the Boards Manager, use the version dropdown next to the Espressif package to select **3.0.7** before installing — do not use the newest version. If a newer version is already installed, select 3.0.7 from that dropdown and click **Install** to downgrade. Uploads can fail or behave oddly on other versions.

Still in the Arduino IDE, open **Tools → Manage Libraries** and install these three (search each name, click Install): **Adafruit BMP280**, **Adafruit NeoPixel**, **TinyGPSPlus**. These are only needed for the vehicle's Sender board.

2 Install Python

Go to **python.org/downloads** and install Python 3. On Windows, tick "Add python.exe to PATH" during install.

Then open a terminal (Command Prompt on Windows, Terminal on Mac) and type:

```
pip install pyserial
```

This installs the one extra piece Python needs to talk to the receiver board.

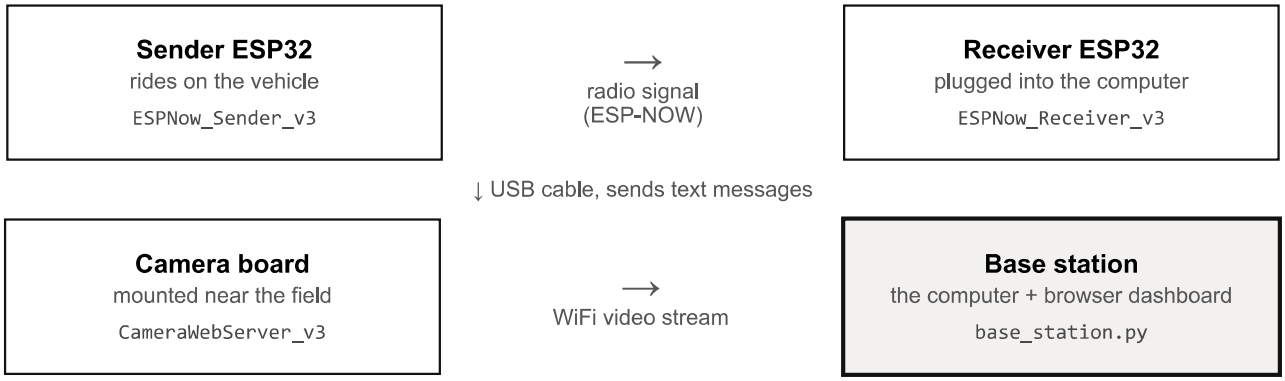
NOT SURE IT WORKED?

Close and reopen the Arduino IDE after installing the ESP32 boards package. In the terminal, typing **python --version** should print a version number, not an error.

PART 03

How the pieces connect

There are four separate programs. Each gets loaded onto a different piece of hardware. Here is the whole picture before you start:



Everything downstream of the receiver — the map, the camera view, the telemetry numbers — is drawn in your web browser by the base station program. You'll set that up in Part 07.

WHICH BOARD IS WHICH, IN THE ARDUINO IDE

PROGRAM FOLDER	GOES ON	BOARD SETTING
`ESPNow_Sender_v3`	Vehicle ESP32	ESP32 Dev Module
`ESPNow_Receiver_v3`	Computer's USB ESP32	ESP32 Dev Module
`CameraWebServer_v3`	Freenove camera board	ESP32 Wrover Module, Partition Scheme: Huge APP (3MB)
`BaseStation`	The computer itself	— (not flashed, just run with Python)

ONE RULE THAT SAVES HEADACHES

All three boards talk over serial at **115200 baud**. If a serial monitor window looks like scrambled text, check the baud rate dropdown in the bottom-right of that window is set to 115200.

Flash the Receiver

Do this first. The receiver needs to exist and print its address before the sender can talk to it.

1. Plug the receiver's ESP32 board into the computer with USB.
2. In the Arduino IDE, open **ESPNow_Receiver_v3/ESPNow_Receiver_v3.ino**.
3. Set **Tools** → **Board** to **ESP32 Dev Module**, and **Tools** → **Port** to whichever port just appeared (unplug/replug if you're unsure which one it is).
4. Click **Upload** (the right-arrow button). Wait for "Done uploading."
5. Open **Tools** → **Serial Monitor** and set its baud rate to **115200**.
6. The receiver prints its own MAC address. **Write this down —** you'll paste it into the sender's code in the next step.

KEEP THIS WINDOW IN MIND

Only one program can use the receiver's USB port at a time. Leave this Serial Monitor open for now if you like, but remember to close it later, before starting the base station in Part 07.

Flash the Sender

This board rides on the vehicle and reads all the sensors. You'll need the MAC address you wrote down in Step 1.

1. In the Arduino IDE, open **ESPNow_Sender_v3/ESPNow_Sender_v3.ino**.
2. Find the line near the top that defines **PEER_MAC** and replace it with the receiver's address from Step 1.
3. Connect the vehicle's ESP32 to the computer with USB. Set **Tools** → **Board** to **ESP32 Dev Module** and pick the matching **Port**.
4. Click **Upload**.
5. Open the **Serial Monitor** at 115200 baud. The board runs a self-test and prints PASS or FAIL for each sensor, with a wiring hint if something fails.
6. Once running, each attempt to send data prints **DELIVERED** or **MISSED**.

IF YOU SEE "MISSED"

Check the receiver board is powered on and that both boards are on WiFi channel 1 (this is already set in the code — don't change it). Double-check the MAC address was copied correctly, with no extra spaces.

Flash the Camera

The camera board streams live video over your WiFi network — it does not need to be plugged into the computer once it's set up.

1. In the Arduino IDE, open **CameraWebServer_v3/CameraWebServer_v3.ino**.
2. Find the **NETWORKS[]** list near the top and add your WiFi name and password. You can list more than one network if needed.
3. Connect the camera board with USB. Set **Tools** → **Board** to **ESP32 Wrover Module**, and **Tools** → **Partition Scheme** to **Huge APP (3MB)** — this board needs both settings correct or the upload will fail.
4. Click **Upload**.
5. Open the **Serial Monitor** at 115200 baud. Once connected to WiFi, it prints a **Video stream** address that looks like `http://<ip-address>:81/stream`. Write this address down.

IMPORTANT

The camera only works on **2.4 GHz** WiFi, not 5 GHz. If it won't connect, it prints a list of networks it can actually see — check your network's name is in that list.

Start the Base Station

This is the program that turns raw data into the live map and dashboard you watch during a run.

1. **Close the receiver's Serial Monitor window** in the Arduino IDE — only one program can use that USB port, and it needs to be free for the base station.
2. Open a terminal (Command Prompt on Windows, Terminal on Mac) and type:

```
pip install pyserial
cd BaseStation
python base_station.py
```

3. Open a web browser and go to **http://localhost:8000**.
4. In the dashboard's **Setup** panel, paste in the camera's video stream address from Step 3.
5. Continue on to Part 08 to enter the field calibration — the dashboard won't place the vehicle correctly on the map until that's done.

"SERIAL: NO PORT FOUND"

This means the receiver is unplugged, or its Arduino Serial Monitor window is still open and holding the port. Unplug/replug the receiver, close that window, and restart base_station.py.

PART 08

Calibrate the field

The dashboard draws the field as a graph. Calibration tells it exactly where that graph sits in the real world, so the vehicle's GPS position lands in the right spot.

Think of the field as graph paper: **(0, 0) is the bottom-left corner**, **x** increases to the right along the long side, and **y** increases going up across the width.

1. Stand at the corner of the field you want to use as the origin, facing down the **long** side, with the field on your **left**.
2. Park the vehicle at that corner and read its lat/lon off the dashboard (or use a phone's GPS).
3. In the Setup panel, enter: **origin latitude/longitude** (the corner you're standing at), **bearing** (the compass direction you're facing), and **length / width** of the field in meters.

Once entered, the vehicle appears at the correct spot on the drawn field, with an arrow showing which way it's pointed and a trail showing where it's been.

VEHICLE DOT MISSING FROM THE MAP?

It needs a GPS fix (the GPS status pill on the dashboard turns green when it has one) **and** a non-zero origin latitude entered in Setup.

PART 09 · OPTIONAL

Waypoints & a destination

SETTING A DESTINATION

In Setup → Destination, enter an x/y point in feet from the origin corner — the same graph coordinates used for waypoints below. A gold crosshair marks the spot on the map, and two telemetry cards, **Dest ΔX** and **Dest ΔY**, count down toward 0 as the vehicle gets closer, turning green once it's within 10 feet on each axis. Click **Clear** to remove the target.

LOADING WAYPOINTS FROM EXCEL

In the Waypoints card, load a **.xlsx** or **.csv** file. Use the first sheet, with column headers in row 1, one waypoint per row. Two header layouts work:

HEADERS	MEANING	TEMPLATE FILE
`name, x_ft, y_ft`	Feet from the origin corner, same as the field graph	`waypoints_example_xy.xlsx`
`name, lat, lon`	GPS coordinates, in degrees	`waypoints_example.xlsx`

Both templates live in the **BaseStation** folder. Optionally, add a second sheet named **field** with key/value rows (`origin_lat, origin_lon, bearing_deg, length_m, width_m`) to auto-fill the calibration from Part 08 when the file loads. The x/y template has no **field** sheet, so loading it never overwrites your existing calibration.

PART 10

Troubleshooting

SENDER KEEPS PRINTING "MISSED"

Is the receiver powered on? Are both boards left on WiFi channel 1 (default, don't change it)? Re-check the PEER_MAC address was copied exactly.

RECEIVER PRINTS NOTHING

It warns after 5 seconds with no packets received — that's normal while you're still setting up the sender. Check the sender first.

"SERIAL: NO PORT FOUND"

The receiver is unplugged, or its Arduino Serial Monitor is still open, holding the port. Close that window and reconnect the USB cable.

CAMERA WON'T JOIN WIFI

It prints a scan of networks it can see — check for typos in your network name. Remember: 2.4 GHz only.

CAMERA IMAGE WORKS, BUT NO VIDEO IN THE DASHBOARD

The computer running the base station must be on the **same** network as the camera. Test the stream address directly in a browser tab first.

VEHICLE DOT MISSING FROM THE MAP

Needs a GPS fix (green GPS pill on the dashboard) and a non-zero origin latitude entered in Setup.

PART 11

Quick reference card

Cut out or photocopy this page and keep it by the base station.

ORDER	BOARD / PROGRAM	BOARD SETTING
1	ESPNow_Receiver_v3 → Receiver ESP32	ESP32 Dev Module
2	ESPNow_Sender_v3 → Vehicle ESP32	ESP32 Dev Module
3	CameraWebServer_v3 → Camera board	ESP32 Wrover Module · Huge APP (3MB)
4	base_station.py → the computer	python base_station.py

All serial monitors: 115200 baud · **Dashboard:** http://localhost:8000 · **Camera stream:** http://<ip>:81/stream

START-UP ORDER EVERY TIME

1. Power the vehicle → 2. Plug in the receiver → 3. Power the camera → 4. Run `python base_station.py` → 5. Open http://localhost:8000